National Qualifications 2017

2017 Computing Science

Advanced Higher

Finalised Marking Instructions

© Scottish Qualifications Authority 2017

The information in this publication may be reproduced to support SQA qualifications only on a non-commercial basis. If it is reproduced, SQA should be clearly acknowledged as the source. If it is to be used for any other purpose, written permission must be obtained from permissions@sqa.org.uk.

Where the publication includes materials from sources other than SQA (secondary copyright), this material should only be reproduced for the purposes of examination or assessment. If it needs to be reproduced for any other purpose it is the centre's responsibility to obtain the necessary copyright clearance. SQA's NQ Assessment team may be able to direct you to the secondary sources.

These marking instructions have been prepared by examination teams for use by SQA appointed markers when marking external course assessments. This publication must not be reproduced for commercial or trade purposes.

General marking principles for Advanced Higher Computing Science

This information is provided to help you understand the general principles you must apply when marking candidate responses to questions in this paper. These principles must be read in conjunction with the detailed marking instructions, which identify the key features required in candidate responses.

(a) Marks for each candidate response must always be assigned in line with these general marking principles and the detailed marking instructions for this assessment.

(b) Marking should always be positive. This means that, for each candidate response, marks are accumulated for the demonstration of relevant skills, knowledge and understanding: they are not deducted from a maximum on the basis of errors or omissions.

(c) If a specific candidate response does not seem to be covered by either the principles or detailed marking instructions, and you are uncertain how to assess it, you must seek guidance from your Team Leader.

(d) Marks should be awarded regardless of spelling as long as the meaning is unambiguous.

(e) Candidates may answer programming questions in any appropriate programming language or pseudocode. Marks should be awarded, regardless of minor syntax errors, as long as the intention of the coding is clear.

(f) Where a question asks the candidate to **describe**, the candidate must provide a statement or structure of characteristics and/or features. This should be more than an outline or a list. It may refer to, for instance, a concept, process, experiment, situation or facts in the context of, and appropriate to, the question. The candidates will normally be required to make the same number of factual/appropriate points as there are marks available for the question.

(g) Where a question asks the candidate to **explain**, marks should only be awarded where the candidate goes beyond a description, for example by giving a reason, or relating cause to effect, or providing a relationship between two aspects. These will be related to the context of the question or a specific area within a question.

(h) Credit should be given where a labelled diagram conveys clearly and correctly the response required by the question.

Detailed marking instructions for each question

Question			Expected answer(s)	Max mark	Additional guidance
1.	(a)		Gantt chart should indicate: • correct list of tasks with duration in hours • concurrent Tasks 1 – 3 followed by Task 4	**2**	1 mark for correct list of tasks with correct duration in days 1 mark for concurrency of Tasks 1 – 3 followed by Task 4
	(b)		A record structure would allow each attribute to be given an appropriate data type. **(1 mark)** This will make it easier to determine the winning distance for each category as no conversion of data type required **(1 mark)**	**2**	1 mark for individual data types 1 mark for importance/relevance in this example *alternatively* 1 mark for exemplification of use made of different data types based on scenario
	(c)	(i)	Recursion	**1**	
		(ii)	A stack is used by in the execution of the search procedure to allow the program to keep track of where it should return control to once a procedure has finished executing **(1 mark)** Whenever the search procedure is called, it is added to the call stack **(1 mark)** Each procedure call or recursion will have an active version in the stack **(1 mark)** Final call returns value to previous version in stack and terminates. This repeats until value displayed **(1 mark)** When the base case is reached, the last procedure call added to the stack is removed and control is passed to the previous procedure call. This repeats until the first procedure call is reached and a value is returned **(1 mark)**	**2**	Award 1 mark description of a stack Award 1 mark for relevance in this situation
		(iii)	This is used to terminate the recursion **(1 mark)** Without this, program tries to call itself indefinitely (or until memory runs out). **(1 mark)** Recursion will terminate when a value of -1 is returned **(1 mark)** **1 mark each for any two**	**2**	Award maximum of 2 marks for any 2 of the following: 1 mark for termination of recursion 1 mark for need to exit recursion 1 mark for termination value 1 mark for base case

Question			Expected answer(s)	Max mark	Additional guidance
1.	(d)		Line 152 SET sLongest [count] TO sLongest [count+1] Line 153 SET sLongest [count+1] TO temp	**2**	1 mark for correct use made of record structure 1 mark for correct use of temp variable to swap array[count] with array[count+1]
	(e)		A queue has a first in/first out principle **(1 mark)** As first on list may not be in the correct category may not provide correct record to add to competitors list **(1 mark)**	**2**	1 mark for FIFO/operation of queue 1 mark for explanation of issue arising in this example
	(f)		For example: Program could be executed using low carbon equipment **(1 mark)** This would reduce the amount of carbon emissions when the code is executed and would be an environmental friendly solution **(1 mark)**	**2**	1 mark for description of any appropriate solution 1 mark for justification in terms of environmental friendliness

<table>
<tr><th colspan="3">Question</th><th>Expected answer(s)</th><th>Max mark</th><th>Additional guidance</th></tr>
<tr><td>2.</td><td>(a)</td><td></td><td>Complete data dictionary is shown below.

<table><tr><th>Field</th><th>Type/Size</th><th>Key</th><th>Constraints / Validation</th></tr><tr><td>userID</td><td>NUMBER
or INTEGER</td><td>PK</td><td>Not null, auto increment (unique)</td></tr><tr><td>title</td><td>VARCHAR (4)
or CHAR(4)</td><td></td><td>One of Mr, Mrs, Miss, Ms</td></tr></table></td><td>3</td><td>1 mark for correct data types (with appropriate sizes) for both fields

1 mark for correct PK

1 mark for constraints for title (restricted choice) with appropriate constraint for userID</td></tr>
<tr><td></td><td>(b)</td><td>(i)</td><td><u>PHP solution</u>

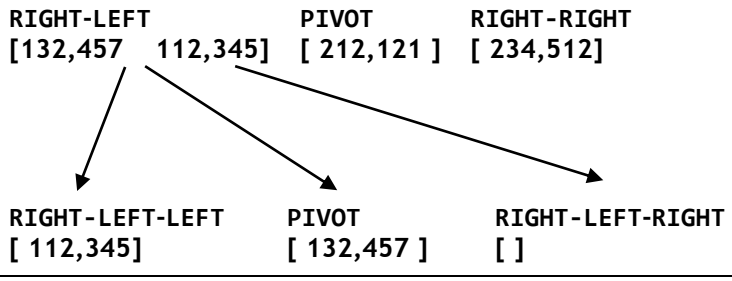
// Assign variables
$title = $_POST['Item1'];
$firstname = $_POST['Item2'];
$lastname = $_POST['Item3'];
$phone = $_POST['Item4'];
$password = $_POST['Item5'];</td><td>1</td><td>1 mark for correct assignment of variables using HTML attributes and POST method</td></tr>
<tr><td></td><td></td><td>(ii)</td><td>// Create connection
$servername = "sn001";
$username = "anon001";
$password = "ap001;
$dbname="RestaurantApp";

$conn = mysql_connect($servername, $username, $password);

Note: use of mysqli_connect requires 4 parameters ($dbname must be included)</td><td>2</td><td>1 mark for correct use made of all three connection details (note that database is not necessary for my_sql connection code)

1 mark for correct connection code</td></tr>
<tr><td></td><td></td><td>(iii)</td><td>// Execute query
$query = "INSERT INTO RegUser (Title, FirstName, LastName, PhoneNo, Password) VALUES ('$Title','$firstname', '$lastname', '$phone', '$password');

mysqli_query($conn, $query);</td><td>3</td><td>1 mark for correct INSERT INTO query with 5 fields named (values must match the named sequence)
1 mark for correct use of database fields (named in part (a) answer) with matching server-side variables
1 mark for correct code use to execute SQL query</td></tr>
</table>

Question			Expected answer(s)	Max mark	Additional guidance
2.	(c)		SELECT title, firstname, lastname, time, sizeofparty FROM RegUser, Reservation WHERE RegUser.userID = Reservation.regUserID AND date="07/01/2017" AND restaurantID=1 ORDER BY time ASC;	**3**	1 mark each for correct – • SELECT and FROM clauses with join • criteria used in WHERE clause (date AND restaurantID) • ORDER BY clause Note: INNER JOIN may be used as alternative to equi-join eg SELECT … FROM regUser INNER JOIN reservation ON regUser.userID = reservation.regUserID WHERE … ORDER BY …
	(d)		RIGHT-LEFT [132,457 112,345] PIVOT [212,121] RIGHT-RIGHT [234,512] RIGHT-LEFT-LEFT [112,345] PIVOT [132,457] RIGHT-LEFT-RIGHT [ ]	**3**	1 mark for correct left sub-arrays 1 mark for correct pivots 1 mark for correct right sub-arrays
					

Question			Expected answer(s)	Max mark	Additional guidance
3.	(a)	(i)	RemoteSensorValve is a subclass of the SensorValve class. It inherits the attributes and methods from the SensorValve (super) class. The subclass can access the attributes and methods of the super class if the permissions have been granted by allowing [public/protected] access. RemoteSensorValve may call any public methods of the SensorValve class as if they were defined within itself.	**2**	Award 1 mark each for any two of the following: • 1 mark for explanation of Subclass/Super class • 1 mark for explanation of inheritance • 1 mark notion of encapsulation
		(ii)	CLASS RemoteSensorValve INHERITS SensorValve { WITH INTEGER currentValue } METHODS CONSTRUCTOR RemoteSensorValve() DECLARE THIS.currentValue INITIALLY 0 END CONSTRUCTOR FUNCTION getCurrentValue() RETURNS INTEGER RETURN THIS.currentValue END FUNCTION END CLASS	**3**	1 mark for inheritance with currentValue 1 mark for (accessor) method that returns currentValue 1 mark for the constructor method Notes • CLASS, METHODS and END CLASS are not required to gain full marks • Other languages are possible and are acceptable alternatives to the Haggis solution
		(iii)	DECLARE rsv1 INITIALLY RemoteSensorValve()	**1**	Note: class name must be correct

Question			Expected answer(s)	Max mark	Additional guidance
3.	(b)		PROCEDURE insertionSort(ARRAY OF RemoteSensorValve rsvArray) FOR index FROM 1 TO 9999 DO SET posn TO index WHILE posn > 0 AND rsvArray[posn-1]. getCurrentValue() > rsvArray[posn].getCurrentValue() SET temp TO rsvArray[posn] SET rsvArray[posn] TO rsvArray[posn-1] SET rsvArray[posn-1] TO temp SET posn TO posn-1 END WHILE END FOR END PROCEDURE	**5**	Award 1 mark each for any 5 of the following: 1 mark for use of array of objects with appropriate indexing 1 mark for appropriate use of getCurrentValue method 1 mark for use of fixed loop to process all elements of the array except the first one 1 mark for use of conditional loop with correct complex condition to control inner loop **OR** 1 mark for use of fixed loop with a step of -1 to control inner loop 1 mark for correct use (inc updating) of posn variable needed to control inner loop 1 mark for swapping correct array element 1 mark for correct comparison of adjacent items in the array

Question			Expected answer(s)	Max mark	Additional guidance
3.	(c)		Imogen's procedure is created in isolation from the rest of the manufacturing application and therefore, component testing can be used to make sure that it works correctly being it is integrated within the larger application. She should write use a stub or driver to activate the subprogram and pass parameter values to it. Component testing focusses on the output of this single procedure and is a form of black box testing: we are interested in the fact that the output is correct and aren't concerned about how the output is produced. A set of test data should be created and expected output should be calculated/computed. The results of applying the test data values to the code should be compared with the output produced to determine whether or not the subprogram is working correctly.	**2**	1 mark for explanation of use made of component testing to test Imogen's subprogram 1 mark for description of how this component testing would be carried out. This may refer to use made of: • stubs/drivers • test data and expected output values • black box testing
	(d)		Many implications could be described. For example: • Fault Diagnostics • Airline safety • Passenger confidence	**2**	1 mark for suggesting an appropriate implication 1 mark for description of that implication for Airflappe Airlines

Question			Expected answer(s)	Max mark	Additional guidance
4.	(a)		Ali would ask stakeholders (parents, teachers, young primary pupils) to list/rank/identify the feelings they find young children using most often; she would ask the target audience (young children) to complete the surveys by saying what their most common emotion or feeling was. Once the surveys had been completed by a large group of the target audience, the results would be analysed to determine the 4 most commonly used emotions for the age group.	**2**	1 mark for source(s) of the information gathered 1 mark for description of details gathered in the survey
	(b)		Many different solutions are possible. One solution is provided below. 1. declare 1D array of integers to store quantity of each image 2. initialise quantity array to 0 3. repeat rows from 0 to 4 4. repeat columns from 0 to 7 5. selectNumber 6. store selected number in array[row, column] 7. end repeat 8. end repeat 5.1 set valid to false 5.2 while not valid 5.3 select random number between 1 and 4 5.4 if quantity of that number <10 then 5.5 add 1 to quantity 5.6 set valid to true 5.7 else 5.8 select random number between 1 and 4 5.9 end if 5.10 end while	**5**	1 mark for appropriate use made of local variables or 1D array to store count of each random number 1 mark for correct use nested loop structure Max 2 marks allocated to check: 1 mark for comparison of counts with 10; 1 mark for taking appropriate action when count exceeds 10 1 mark for correct storage in 2-D array

Question			Expected answer(s)	Max mark	Additional guidance
4.	(c)		Assumptions: • linked list is implemented as an array of nodes which is passed to the procedure • RECORD Nodes IS { STRING ending, INTEGER pointer } • head of list is element 0 • pointer of tail of list contains value 999 PROCEDURE outputToFile (ARRAY of Nodes array) DECLARE index INITIALLY 0 OPEN file "happyFeelings.txt" WHILE head <> 999 DO SEND array.ending[index] TO file "happyFeelings.txt" SET index TO array.pointer[index] END WHILE CLOSE file "happyFeelings.txt" END PROCEDURE	**4**	1 mark for file handling (OPEN, SEND, WRITE, CLOSE) Max 3 marks for handling linked list: 1 mark for traversing the list 1 mark for use of head/tail 1 mark for writing node data value to file Note: many alternative solutions are possible
	(d)		Creating Thread C means that Threads A & B would not be interrupted when the backup started. As a result, the user experience wouldn't be slowed down/ interrupted while the backup was performed. If Thread C wasn't programmed then commands from the keyboard would be ignored until the backup was complete.	**2**	Award max 2 marks for any 2 of the following: 1 mark no interruption/time split between only two threads 1 mark keyboard commands when ignored would result in sluggish performance when using the keyboard. 1 mark for appropriate description of multiple threads Note: full marks can only be allocated if answer relates back to the scenario provided

Question			Expected answer(s)	Max mark	Additional guidance
4.	(e)		Copyright Ali will automatically gain Copyright protection for any original artwork she has created for the program. Ali will automatically gain Copyright protection on the program/source code she has written. Copyright protection protects Ali's work and stops others from using it without her permission. Copyright prevents people from: • copying her work • distributing copies of it, whether free of charge or for sale • renting or lending copies of her work • performing, showing or playing her work in public • making an adaptation of her work • putting it on the internet Design Right Ali will automatically gain Design Right protection on the design of her program. Design right automatically protects Ali's design for 10 years after it was first sold or 15 years after it was created - whichever is earliest.	**2**	1 mark for either copyright or design right 1 mark for accurate description of named protection

[END OF MARKING INSTRUCTIONS]